



Addis Ababa University

College Of Technology And Built Environment

School of Information Technology and Engineering

Fundamentals of Artificial Intelligence

Implementation of Informed Search Algorithms

Section-4

Members

1. Anania Girma	UGR/1373/16
2. Ermias Haimanot	UGR/8502/16
3. Mekdelawit Alemayehu	UGR/9271/16
4. Mihret Tena	UGR/4204/16
5. Teumay Werashe	UGR/8281/16
6. Yeabsra Seyoum	UGR/2175/16

Implementation of Informed Search Algorithms

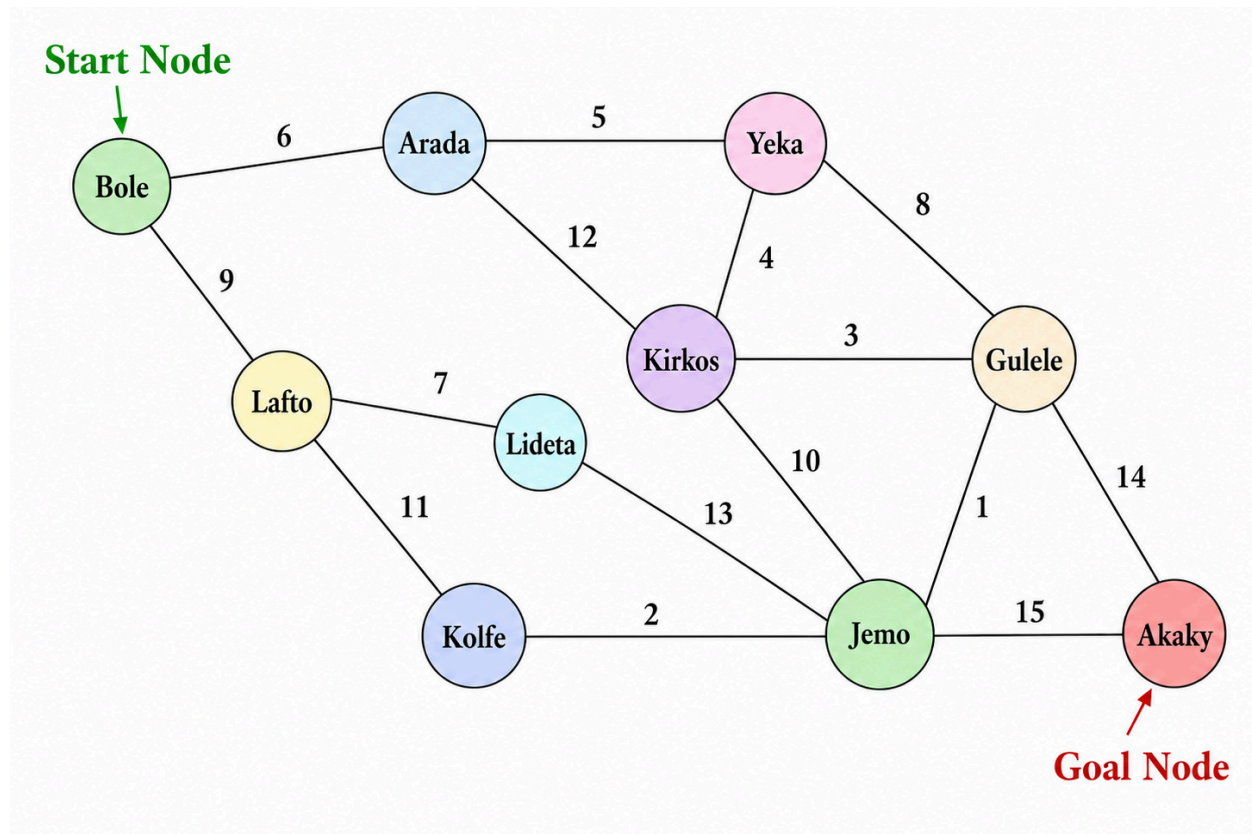


Figure 1: Weighted undirected graph of 10 Addis Ababa sub-cities.

1. Introduction

This report analyzes the performance and behavior of two informed search algorithms: **Greedy Best-First Search (GBFS)** and **A* Search**. The algorithms were tested on a weighted undirected graph representing 10 sub-cities in Addis Ababa, with the objective of finding the most efficient path from **Bole** (Start) to **Akaky** (Goal).

2. Graph Data and Heuristics

The environment consists of 10 nodes with specific travel costs between them. The optimal path based on true edge costs is Bole → Arada → Yeka → Kirkos → Gulele → Akaky with a total cost of 32.

Table 1: Edge Weights

From	To	Cost
Bole	Arada	6
Bole	Lafto	9
Arada	Yeka	5
Arada	Kirkos	12
Lafto	Lideta	7
Lafto	Kolfe	11
Yeka	Kirkos	4
Yeka	Gulele	8
Kirkos	Gulele	3
Kirkos	Jemo	10
Lideta	Jemo	13
Kolfe	Jemo	2
Gulele	Jemo	1
Gulele	Akaky	14
Jemo	Akaky	15

Table 2: Heuristic Values $h(n)$

Node	Original	Good	Misleading
Bole	32	28	10
Arada	26	22	50
Lafto	24	20	5
Yeka	21	18	60
Kirkos	17	15	55
Lideta	28	14	3
Kolfe	17	2	40
Gulele	14	13	50
Jemo	15	14	1
Akaky	0	0	0

3. Analysis and Comparison

The algorithms were evaluated based on their ability to reach the goal, the optimality of the path found, and the search efficiency (nodes expanded).

3.1 Efficiency and Expansion

GBFS generally reached the goal faster in terms of nodes expanded. For the original heuristic, **GBFS** expanded only 5 nodes compared to 6 nodes for A*. **GBFS** achieves this by prioritizing nodes purely based on **$h(n)$** , committing to the direction that appears closest to the goal without revisiting alternatives.

3.2 Optimality

A* consistently produced the optimal solution under admissible heuristics (Original and Good), finding the path with a cost of **32**. In contrast, **GBFS** returned a path cost of **37** under the same conditions. A*'s optimality is guaranteed by its evaluation function **$f(n) = g(n) + h(n)$** , which tracks actual costs already paid alongside heuristic estimates.

Table 3: Summary of Results

Heuristic	Algorithm	Path Cost	Nodes Expanded
Original	GBFS	37	5
Original	A*	32	6
Good	GBFS	37	6
Good	A*	32	9
Misleading	GBFS	44	5
Misleading	A*	44	5

4. Key Observations

Observation 1 : Original Heuristic

- GBFS expanded **5 nodes** and chose the southern route based on lower heuristic values, resulting in a path cost of **37**.
- A* expanded **6 nodes** and used both path cost (**$g(n)$**) and heuristic (**$h(n)$**), finding a cheaper northern route with cost **32**.
- One extra node expansion allowed A* to save **5 units of path cost**.

Observation 2 : Good Heuristic

- The Good heuristic is admissible but **looser than the Original** (which equals true distances).
- A* expanded **9 nodes** while carefully verifying whether the southern route was actually optimal, eventually still finding the cheaper northern path.
- GBFS again followed the southern route and obtained cost **37**.
- A looser admissible heuristic makes A* work harder : closer to the true cost means fewer expansions.

Observation 3 : Misleading Heuristic

- Both GBFS and A* were misled by unrealistically low heuristic values.
- They expanded only **5 nodes** but selected the worst path with cost **44**.
- A* lost its optimality because the heuristic was **not admissible**, causing incorrect path evaluation.
- Fewer expansions in this case indicate **poor search quality rather than efficiency**.

```
PS C:\Users\Mekdelawit\AI_GBFS_A-star> python experimental_exploration.py
EXPERIMENTAL EXPLORATION & TESTING
Graph: Addis Ababa sub-cities | Bole -> Akaky
HEURISTIC SET: ORIGINAL

h(n) values:
Bole      h = 32
Arada     h = 26
Lafto     h = 24
Yeka      h = 21
Kirkos    h = 17
Lideta    h = 28
Kolfe     h = 17
Gulele    h = 14
Jemo      h = 15
Akaky     h = 0

[GBFS]
Path Found      : Bole -> Lafto -> Kolfe -> Jemo -> Akaky
Total Path Cost : 37
Nodes Expanded  : 5 (Bole -> Lafto -> Kolfe -> Jemo -> Akaky)

[A*]
Path Found      : Bole -> Arada -> Yeka -> Kirkos -> Gulele -> Akaky
Total Path Cost : 32
Nodes Expanded  : 6 (Bole -> Arada -> Yeka -> Kirkos -> Gulele -> Akaky)
```

Screenshot 1: Execution output : Original heuristic (GBFS & A*)

```

HEURISTIC SET: GOOD


---


h(n) values:
  Bole      h = 28
  Arada     h = 22
  Lafto     h = 20
  Yeka      h = 18
  Kirkos    h = 15
  Lideta    h = 14
  Kolfe     h = 2
  Gulele    h = 13
  Jemo      h = 14
  Akaky     h = 0

[GBFS]
  Path Found      : Bole -> Lafto -> Kolfe -> Jemo -> Akaky
  Total Path Cost : 37
  Nodes Expanded  : 6 (Bole -> Lafto -> Kolfe -> Lideta -> Jemo -> Akaky)

[A*]
  Path Found      : Bole -> Arada -> Yeka -> Kirkos -> Gulele -> Akaky
  Total Path Cost : 32
  Nodes Expanded  : 9 (Bole -> Arada -> Lafto -> Kolfe -> Yeka -> Kirkos -> Lideta -> Gulele -> Akaky)

```

Screenshot 2: Execution output : Good heuristic (GBFS & A)*

```

HEURISTIC SET: MISLEADING


---


h(n) values:
  Bole      h = 10
  Arada     h = 50
  Lafto     h = 5
  Yeka      h = 60
  Kirkos    h = 55
  Lideta    h = 3
  Kolfe     h = 40
  Gulele    h = 50
  Jemo      h = 1
  Akaky     h = 0

[GBFS]
  Path Found      : Bole -> Lafto -> Lideta -> Jemo -> Akaky
  Total Path Cost : 44
  Nodes Expanded  : 5 (Bole -> Lafto -> Lideta -> Jemo -> Akaky)

[A*]
  Path Found      : Bole -> Lafto -> Lideta -> Jemo -> Akaky
  Total Path Cost : 44
  Nodes Expanded  : 5 (Bole -> Lafto -> Lideta -> Jemo -> Akaky)

```

Screenshot 3: Execution output : Misleading heuristic (GBFS & A)*

COMPARISON SUMMARY TABLE			
Heuristic	Algo	Path Cost	Nodes Expanded

ORIGINAL	GBFS	37	5
ORIGINAL	A*	32	6
GOOD	GBFS	37	6
GOOD	A*	32	9
MISLEADING	GBFS	44	5
MISLEADING	A*	44	5
PS C:\Users\Mekdelawit\AI_GBFS_A-star>			

Screenshot 4: Comparison summary table : all heuristics & algorithms

5. Conclusion

While **GBFS** offers lower computational overhead by expanding fewer nodes, it frequently sacrifices path quality. A* remains the superior choice for finding the shortest path, provided the heuristic remains admissible. This experiment underscores that a low expansion count is not always a sign of efficiency; in cases with misleading heuristics, it can signal a failure to explore better alternatives.